

Weekly Diary – Week 5

Internship Organization: Vikaspedia

Duration: 15/02/2026 – 21/02/2026

Objective of the Week

The objective of this week was to integrate an Artificial Intelligence (AI) model with the chatbot system and enable it to generate answers for user queries based on the specific content available on the webpage. Until this stage, the chatbot interface had already been developed, but it was not yet capable of generating intelligent responses. Therefore, the focus of this week was on connecting the chatbot with an AI API and implementing the logic required for processing user questions.

Another important task was to extract the relevant textual content from the webpage dynamically so that the AI model could use it as context while generating responses. This ensured that the chatbot answers were related to the webpage content rather than producing general responses. The work also involved building the prompt structure, integrating API requests with the frontend, and improving the user experience during response generation.

Date – 15 February

On the first day of the week, I conducted research on different AI APIs that could be integrated into the chatbot system to generate responses for user queries. Since the project required a solution that was easy to integrate and accessible within a free usage tier, I explored several available AI services.

Some of the APIs that were examined included Hugging Face Inference API, OpenRouter models, and Google Gemini API. I compared these options based on factors such as response quality, availability of documentation, ease of integration with JavaScript applications, and limitations in the free usage plans. After reviewing the advantages and limitations of each service, I shortlisted the most suitable API for the chatbot integration.

Date – 16 February

On the second day, I studied the documentation of the selected AI API in detail. Understanding the documentation was important for learning how to properly send requests and receive responses from the model.

During this process, I learned about the authentication methods used by the API, the format of request payloads, and the structure of the responses returned by the AI model. I also explored how prompts should be constructed in order to guide the AI model toward generating meaningful and context-aware responses. Additionally, I reviewed how asynchronous JavaScript functions such as `fetch()` or `async/await` could be used to send API requests and process responses efficiently within the React application.

Date – 17 February

On the third day, I implemented functionality to dynamically extract the textual content from the webpage using JavaScript DOM manipulation methods. This step was necessary because the chatbot needed to understand the information available on the page before answering user questions.

I identified the main content section of the webpage and extracted the text from it while ignoring unnecessary HTML elements such as navigation menus, buttons, or scripts. After extracting the content, I cleaned the text by removing extra spaces, formatting issues, and unwanted characters. This cleaned content would later be used as contextual input for the AI model.

Date – 18 February

On the fourth day, I developed the prompt generation logic required for the chatbot system. The prompt is the input that is sent to the AI model, and its structure plays an important role in determining the quality of the generated responses.

I created a prompt format that combined two important elements: the extracted webpage content and the user's question. By providing both pieces of information together, the AI model could better understand the context and produce answers that were directly related to the webpage. The prompt was structured in a clear and organized way to ensure that the AI system interpreted the information correctly.

Date – 19 February

On the fifth day, I integrated the AI API with the React-based chatbot interface. This involved implementing asynchronous API requests that send the constructed prompt to the selected AI model and receive the generated response.

Once the response was received from the API, it was displayed inside the chatbot message container so that users could read the answer in the chat interface. I tested the integration by sending multiple

example questions and verified that the chatbot was able to generate relevant responses based on the provided context.

Date – 20 February

On the sixth day, I focused on improving the overall user experience of the chatbot interface. Since API responses may sometimes take a few seconds to arrive, it was important to inform users that the system was processing their request.

To address this, I implemented a loading indicator in the chat interface. A message such as “Thinking...” or a spinner animation was displayed while waiting for the AI response. This improvement made the interaction feel smoother and prevented users from assuming that the chatbot had stopped working.

Date – 21 February

On the final day of the week, I carried out detailed testing and debugging of the chatbot system. I asked various types of questions related to the webpage content to verify whether the chatbot was generating responses based on the provided context.

During this testing phase, I also fixed minor issues related to message formatting and API response handling. Ensuring proper formatting of responses helped make the conversation clearer and more readable for users.